



Tree Data Structure

The Complete Beginner-to-Pro Guide

Master this — crack 50% of your tech interviews

□ 8 Tree Types

↗ All Operations

□ Big Tech Use Cases

1. What Is a Tree?

From linear to non-linear — the game changes here

Linear vs Non-Linear — The Key Difference

Every data structure you have studied so far — Array, Linked List, Stack, Queue — is LINEAR. This means to get from one element to another, you must pass through every element in between. Like a single road with no shortcuts.

A Tree is NON-LINEAR. From one node you can branch out in multiple directions. You can jump from node 1 directly to node 4, then to node 10. No forced path. This is what makes trees incredibly powerful.

```

LINEAR (Array/LinkedList/Stack/Queue):
[1] --> [2] --> [3] --> [4] --> [5] --> [6] --> [7]
To reach 7 from 1, you MUST pass through 2,3,4,5,6

```

```

NON-LINEAR (Tree):
      [1]
     /  \
    [2]  [3]
   /  \  \
  [4] [5] [6]
           \
          [7]
From 1, jump to 3, then straight to 7. Done!

```

Interview Gold:

Master Trees = crack ~50% of all technical interview questions at Google, Meta, Amazon, Apple, Microsoft.

Binary Trees, BSTs, and Tree traversals appear in almost every FAANG coding round.

Real-World Analogy — Your Family Tree □□□□

Think of your family tree. Your great-great-grandparents are at the top (root). Their children are below. Each of those children has children. It branches out, level by level, until you reach people alive today (leaf nodes). That is exactly what a Tree data structure looks like.

```

Your Family Tree:
      [Great-Grandpa]      <-- ROOT (Level 0)
     /      \
  [Grandpa]  [Granduncle] <-- Level 1
   /      \

```

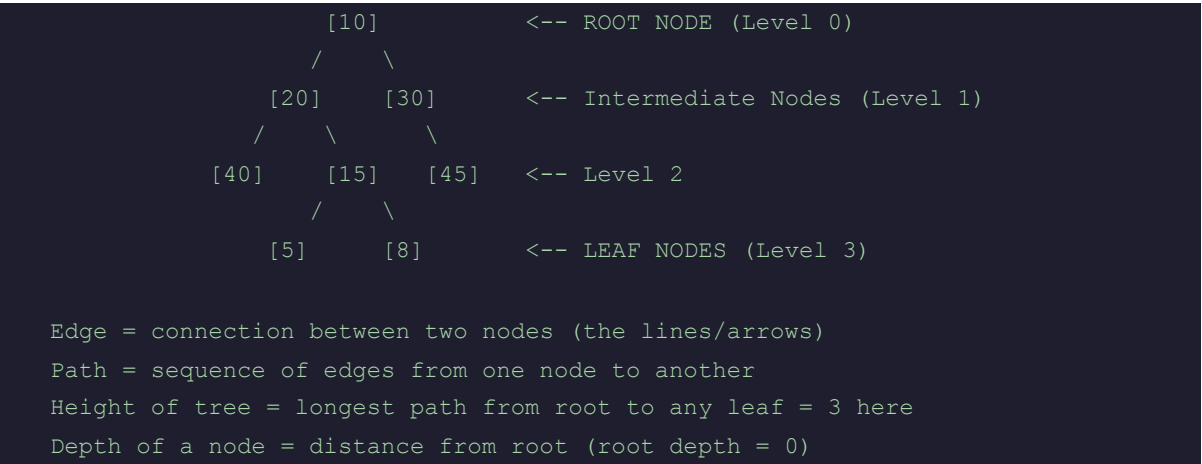
```
    [Dad]      [Uncle]                <-- Level 2 (Intermediate)
  /      \
[You] [Sibling]                <-- Level 3 (LEAF nodes)
```

```
Rules: Go from parent to child (top to bottom)
       Cannot go from child to parent directly
       (Need Stack or Recursion to go upward!)
```

2. Tree Terminology

The vocabulary every developer must know

Key Terms with Examples



Term	Simple Definition	Example from diagram above
Root Node	The very top node. Has children but NO parent	Node 10
Leaf Node	The very bottom nodes. Have a parent but NO children	Nodes 40, 5, 8, 45
Intermediate Node	Any node between root and leaves. Has both parent AND children	Nodes 20, 30, 15
Parent Node	A node that has at least one child below it	Node 20 is parent of 40 and 15
Child Node	A node directly below a parent	Nodes 40 and 15 are children of 20
Siblings	Nodes sharing the same parent	20 and 30 are siblings (parent = 10)
Subtree	A node + all its descendants below it	Node 20 with 40, 15, 5, 8 = a subtree
Level / Depth	How far down from root (root = Level 0)	Node 40 is at Level 2
Height of Tree	Longest path from root to any leaf	4 levels → Height = 3
Edge	The connection/link between two nodes	Line connecting 10 to 20

Term	Simple Definition	Example from diagram above
Degree of Node	Number of children a node has	Node 20 has degree 2

□ **Important Rule:**

You can ONLY go from Parent → Child (top to bottom) by default.

To go from Child → Parent (bottom to top), you need to use RECURSION or a STACK.

This is why recursion is so closely paired with tree problems!

□ **Fun Fact:**

In a real-world tree: the ROOT is on the GROUND and LEAVES are at the TOP.

In a Tree Data Structure: it is UPSIDE DOWN — ROOT is at TOP, LEAVES are at BOTTOM!

3. Types of Trees

8 types — each with a specific purpose

3.1 Binary Tree

The most common tree type. Each node can have at most 2 children — left child and right child. It can have 0, 1, or 2 children. No more.

Binary Tree Example:

[10]

/ \

[20]

[30]

/ \

[40]

[45]

Node 10 → 2 children (20 and 30) ✓

Node 20 → 1 child (40 only) ✓

Node 30 → 1 child (45 only) ✓

Node 40 → 0 children (leaf) ✓

Any node having 3+ children? X NOT allowed

Operation	Best Case	Average Case	Worst Case	Space
Search	O(1) — root	O(n)	O(n) — skewed	O(n)
Insert	O(1) — root empty	O(n)	O(n) — skewed	O(n)
Delete	O(1)	O(n)	O(n)	O(n)
Traversal	O(n)	O(n)	O(n)	O(h) — stack space

Who Uses Binary Trees?

Company	Use Case
Google	Expression parsing in Google Sheets — formula trees are binary trees
Microsoft	Abstract Syntax Trees (AST) in Visual Studio compiler — code parsed as binary trees
Amazon	Decision trees in recommendation engine — binary branches for user choices
Linux Kernel	Red-Black Tree (a self-balancing binary tree) used in process scheduling

3.2 Binary Search Tree (BST)

A Binary Tree with one extra rule: the LEFT subtree always contains values SMALLER than the parent, and the RIGHT subtree always contains values LARGER. This rule makes searching incredibly fast — $O(\log n)$ instead of $O(n)$!

BST Example:

```

      [50]
     /  \
   [30]  [70]
  /  \  /  \
 [20] [40] [60] [80]

```

Rule Check:

Node 50 → Left subtree (30,20,40) all < 50 ✓

Node 50 → Right subtree (70,60,80) all > 50 ✓

Node 30 → Left(20) < 30 < Right(40) ✓

Searching for 60:

Start at 50 → 60 > 50 → go RIGHT to 70

70 → 60 < 70 → go LEFT to 60 → FOUND! ✓

Only 3 comparisons instead of scanning all 7!

Operation	Best Case	Average Case	Worst Case	Space	Note
Search	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$	Worst = skewed tree (like linked list)
Insert	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$	h = height of tree
Delete	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$	Need to restructure after delete
Min/Max	$O(1)$	$O(\log n)$	$O(n)$	$O(1)$	Min = leftmost, Max = rightmost
Inorder Traversal	$O(n)$	$O(n)$	$O(n)$	$O(h)$	Gives SORTED output!

❏ Secret Superpower:

In-order traversal of a BST gives you all elements in SORTED ORDER — for FREE!

This is why BSTs are used in database indexing and sorted data retrieval.

□ Who Uses BSTs?

Company	Use Case
MySQL / PostgreSQL	B-Tree indexes (extension of BST) — powers all database WHERE queries
Google	Search index — sorted term lookup using tree-based indexes
MongoDB	WiredTiger storage engine uses B-Trees for document indexing
Linux	Red-Black BST for managing virtual memory areas (VMA) in the kernel
Java SDK	TreeMap and TreeSet — BST-backed sorted collections used everywhere

3.3 Strict Binary Tree (Full Binary Tree)

Every node must have EXACTLY 0 or 2 children. Having just 1 child is NOT allowed. Either a node is a leaf (0 children) or it has both left AND right children (2 children).

Strict Binary Tree:

```

      [10]
     /  \
    [20] [30]    <- both have 2 children ✓
   /  \ /  \
  [40][50][60][70] <- all are leaves (0 children) ✓

```

NOT Strict (invalid):

```

      [10]
     /  \
    [20] [30]
   /
  [40]    <- Node 20 has only 1 child = INVALID X

```

Formula: If a Strict BT has L leaf nodes,
Total nodes = $2L - 1$

Operation	Best	Average	Worst	Space
Search	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
Insert	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
Delete	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$

Who Uses Strict Binary Trees?

Company / Field	Use Case
Huffman Encoding (used everywhere)	Data compression — every internal node has exactly 2 children. Used in ZIP, JPEG, MP3
Decision Tree ML Models	Scikit-learn decision trees — each decision node branches into exactly 2 outcomes
Compiler Design	Expression trees — binary operators always have exactly 2 operands
Game Theory	Game trees for 2-player games (chess, tic-tac-toe) — each move has 2 branches

3.4 Complete Binary Tree

All levels are completely filled EXCEPT possibly the last level. And the last level must be filled from LEFT to RIGHT — you cannot skip to the right side without filling the left first. Also, the previous level must be completely full before adding to the next.

```

Complete Binary Tree:          NOT Complete:
      [10]                      [10]
      /  \                    /  \
     [20] [30]              [20] [30]
      /  \  /                /  \  \
    [40] [50] [60]         [40] [50] [70]
                                (right filled before left = WRONG)

Left to Right Rule:
[10]→[20]→[30]→[40]→[50]→[60] = correct order

KEY DIFFERENCE from Binary Tree:
Binary Tree: no rules about shape
Complete BT: strict left-to-right filling, level by level

```

Operation	Best	Average	Worst	Space	Note
Search	$O(1)$	$O(n)$	$O(n)$	$O(n)$	Not inherently sorted
Insert	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$	Always at next left-to-right position
Delete	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Replace with last node
Heapify	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	Fundamental heap operation

□ Most Important:

Complete Binary Trees are the FOUNDATION of the HEAP data structure.

Heaps power: Priority Queues, Dijkstra's algorithm, Heap Sort, and OS schedulers.

This is one of the most practically used tree types in production systems!

□ Who Uses Complete Binary Trees?

Company	Use Case
Google	Priority queue in Dijkstra's algorithm for Google Maps routing — heap-based
Amazon	AWS task priority scheduler — min-heap (complete BT) for job scheduling
Operating Systems	Linux kernel heap for timer management and process scheduling
Python	heapq module — complete binary tree stored as an array — used in every Python program
Java	PriorityQueue class — complete BT implementation used in Android/server apps

3.5 Skewed Binary Tree

All nodes are connected in a single direction — like a linked list. If every node only has a RIGHT child, it is RIGHT SKEWED. If every node only has a LEFT child, it is LEFT SKEWED. Each level has exactly one node.

```
Left Skewed:      Right Skewed:
[10]               [10]
/                  \
[20]               [20]
/                  \
[30]               [30]
/                  \
[40]               [40]
```

Essentially a Linked List — worst case for most tree operations!
All operations degrade to $O(n)$ — no branching advantage at all.

Operation	Best	Average	Worst	Note
Search	$O(1)$	$O(n)$	$O(n)$	No branching, must traverse all nodes linearly
Insert	$O(1)$	$O(n)$	$O(n)$	Must reach the end of the chain
Delete	$O(1)$	$O(n)$	$O(n)$	Must find the node first — takes $O(n)$
Height	$O(n)$	$O(n)$	$O(n)$	Height = $n-1$, defeating tree's purpose

❏ **WARNING — This is a Performance Disaster:**

A skewed tree is THE WORST case for a BST.

Inserting sorted data (1,2,3,4,5...) into a BST creates a right-skewed tree.

This is why we use Self-Balancing Trees (AVL, Red-Black) — they prevent skewing.

Google, Amazon, Linux all use self-balancing trees to avoid this exact problem!

3.6 Degenerate Tree

Like a Skewed tree — each level has exactly ONE node. But unlike skewed trees, the direction ALTERNATES: left, then right, then left, then right. It makes a zigzag shape instead of a straight line. Still performs like a linked list — $O(n)$ for everything.

```
Degenerate Tree (Zigzag pattern):
```

```
[10]
/
[20]
 \
  [30]
   /
  [40]
   \
   [50]
```

One node per level. Zigzag = Degenerate Tree.

Performance: Same as Skewed — all $O(n)$. Avoid in production!

Operation	Time Complexity	Space Complexity	Note
Search	O(n)	O(n)	Must check every node — no shortcut
Insert	O(n)	O(n)	Must traverse to bottom
Delete	O(n)	O(n)	Find + remove = O(n)

3.7 Extended Binary Tree

Take any Binary Tree and force it to become a STRICT binary tree by adding dummy (fake/null) nodes wherever a node has only 1 child. These dummy nodes are called external nodes, and the original nodes are internal nodes. This concept is used in compiler design and Huffman coding analysis.

Original (not strict):

Extended (strict with dummy nodes):

```
graph TD
    10["[10]"] -- / --> 20["[20]"]
    10 -- \ --> 30["[30]"]
    20 -- / --> 40["[40]"]
    30 -- \ --> null1[" "]
    40 -- \ --> null2[" "]
    null1 -- / --> star1["[*]"]
    null1 -- \ --> star2["[*]"]
    star1 -- / --> star3["[*]"]
    star1 -- \ --> star4["[*]"]
    star2 -- / --> star5["[*]"]
    star2 -- \ --> star6["[*]"]
```

[*] = dummy nodes added to make every node have 0 or 2 children
Internal nodes = original nodes (10, 20, 30, 40)
External nodes = dummy nodes (*)

Property: If there are n internal nodes,
there will always be n+1 external nodes!

❑ Who Uses Extended Binary Trees?

Field / Company	Use Case
Huffman Compression (ZIP, JPEG, MP3)	Huffman tree is an extended BT — analysis of path lengths uses external nodes
Compiler Theory	Expression trees extended to analyze operator/operand relationships
Information Theory	Analyzing minimum path lengths and entropy in coding theory

3.8 Generic Tree (N-ary Tree)

A node can have ANY number of children — not just 2. There is no restriction on the number of children. However, this creates a memory allocation challenge: if you don't specify the maximum number of children upfront, it is hard for the system to allocate the right amount of memory.

```
Generic Tree (N-ary):  
    [CEO]  
    /  |  |  \  
    [CTO] [CFO] [CMO] [COO]    <- 4 children!  
    /  \  |  
    [Dev] [QA] [Finance]  
  
N-ary Tree where N can be anything.  
  
Memory Challenge:  
If N is unknown at compile time,  
how do you allocate space for children pointers?  
Solution: Use dynamic arrays or linked list of children.
```

□ Who Uses Generic Trees?

Company / Use Case	How
Operating Systems	File system directory tree — a folder can have unlimited subfolders (N children)
Google Chrome	DOM tree — each HTML element can have unlimited child elements
Organization Charts	Company org charts — a manager can have any number of reports
JSON/XML Parsers	Nested JSON objects form a generic tree — each object can have N keys
React / Vue.js	Component tree — each component can have any number of child components

□ 4. Tree Representation in Memory

How trees are actually stored — Array vs Linked List

4.1 Array Representation

We store the tree in a 1D array level by level (Level 0 first, then Level 1, then Level 2...). Within each level, left node goes before right node. This gives us a predictable index-based formula.

```
Tree:
      [10]
     /   \
  [20]   [30]
 /  \   /  \
[40][15][25][45]

Array:
Index: 0  1  2  3  4  5  6
Value: [10][20][30][40][15][25][45]
```

RULES:

- 1) First store Level 0 (root), then Level 1, then Level 2...
- 2) Within each level: LEFT child first, then RIGHT child

FINDING CHILDREN (given parent at index i):

Left child = $\text{arr}[2*i + 1]$

Right child = $\text{arr}[2*i + 2]$

FINDING PARENT (given child at index i):

Parent index = $\text{floor}((i - 1) / 2)$

Example: Node 10 is at index 0

Its left child = $\text{arr}[2*0+1] = \text{arr}[1] = 20$ ✓

Its right child = $\text{arr}[2*0+2] = \text{arr}[2] = 30$ ✓

Node 20 is at index 1

Its left child = $\text{arr}[2*1+1] = \text{arr}[3] = 40$ ✓

Its right child = $\text{arr}[2*1+2] = \text{arr}[4] = 15$ ✓

□ Array Representation Problem:

Array representation ONLY works perfectly for COMPLETE Binary Trees.

If a tree is NOT complete (has missing nodes in middle),

you must add dummy (None) values to fill gaps — WASTING memory.

Example: A tree with 10 nodes but missing nodes at level 3 might

need an array of size 30+ just to maintain the correct indices!

4.2 Linked List Representation

Each node is a struct/object with 3 fields: data, left pointer, right pointer. Nodes are connected via pointers — no wasted memory for empty positions. This is the most common way to implement trees in practice.

Each Node looks like:

```
+-----+-----+-----+
|  LEFT  |  DATA  |  RIGHT  |
| pointer|          | pointer |
+-----+-----+-----+
```

```
class Node:
    def __init__(self, data):
        self.data = data    # the value
        self.left = None    # pointer to left child
        self.right = None   # pointer to right child
```

Tree in memory:

```
[None|10|-->]          <- root node, no left child shown
|
+--> [None|20|-->] --> [None|40|None]
|      |
|      +--> [None|15|None]
|
+--> [None|30|-->] --> [None|45|None]
```

Linked list = dynamic size, no wasted memory slots.

But: you CANNOT go from child back to parent!

Solution: Use RECURSION (the call stack acts as your path back up the tree automatically).

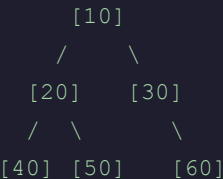
Feature	Array Representation	Linked List Representation
Memory	Wastes memory for incomplete trees	No waste — only allocate what you need
Access speed	$O(1)$ — direct index formula	$O(n)$ — must traverse from root
Works best for	Complete Binary Trees / Heaps	All tree types
Flexibility	Fixed size — hard to grow	Dynamic — grows and shrinks easily
Used in	Heap data structure, priority queues	BST, AVL, Red-Black tree implementations
Navigate to parent	Easy: $\text{floor}((i-1)/2)$	Need recursion or parent pointer

5. Tree Traversal Algorithms

How to visit every node — 4 strategies

Traversal means visiting every node in the tree exactly once in a specific order. There are 4 main ways to do this:

Tree used for all traversal examples below:



5.1 Inorder Traversal (Left → Root → Right)

Visit LEFT subtree first, then the ROOT, then RIGHT subtree. For a BST, this always gives SORTED output — one of the most important properties!

Pattern: LEFT → ROOT → RIGHT

Step by step for tree above:

- 1. Go left from 10 → visit 20's left subtree
- 2. Go left from 20 → visit 40 (leaf, no children)
- 3. Visit 40 → print [40]
- 4. Back to 20 → print [20]
- 5. Visit 20's right → print [50]
- 6. Back to 10 → print [10]
- 7. Go right to 30 → 30 has no left child
- 8. Print [30]
- 9. Visit 30's right → print [60]

Result: 40 → 20 → 50 → 10 → 30 → 60

Python Code:

```
def inorder(node):
    if node is None: return
    inorder(node.left)    # go left first
    print(node.data)      # then print
    inorder(node.right)   # then go right
```

Property	Value
Time Complexity (Best)	O(n) — must visit every node

Property	Value
Time Complexity (Avg)	$O(n)$
Time Complexity (Worst)	$O(n)$
Space Complexity	$O(h)$ — h = height of tree (recursion call stack). $O(\log n)$ balanced, $O(n)$ skewed
Key Property	On a BST: gives output in SORTED ASCENDING ORDER
Used for	Sorting, expression evaluation, retrieving sorted data from BST

5.2 Preorder Traversal (Root → Left → Right)

Visit the ROOT first, then LEFT subtree, then RIGHT subtree. Useful for copying a tree or serializing it to store in a file/database.

```
Pattern: ROOT → LEFT → RIGHT
```

Step by step:

1. Visit root → print [10]
2. Go left → print [20]
3. Go left of 20 → print [40]
4. 40 is leaf → backtrack to 20's right
5. Print [50]
6. Backtrack to 10's right → print [30]
7. 30 has no left → go right → print [60]

```
Result: 10 → 20 → 40 → 50 → 30 → 60
```

Python Code:

```
def preorder(node):  
    if node is None: return  
    print(node.data)      # print FIRST  
    preorder(node.left)   # then left  
    preorder(node.right)  # then right
```

Property	Value
Time Complexity (All cases)	$O(n)$ — visits every node exactly once
Space Complexity	$O(h)$ — recursion depth equals tree height
Key Property	First element printed is always the ROOT
Used for	Copying/cloning trees, serializing trees to file, prefix expression evaluation

5.3 Postorder Traversal (Left → Right → Root)

Visit LEFT subtree, then RIGHT subtree, then the ROOT last. Useful for deleting a tree (delete children before parent) or evaluating postfix expressions.

Pattern: LEFT → RIGHT → ROOT

Step by step:

1. Go left to 20 → go left to 40
2. 40 is leaf → print [40]
3. Back to 20 → go right to 50
4. 50 is leaf → print [50]
5. Both children done → print [20]
6. Back to 10 → go right to 30
7. 30 has no left → go right to 60
8. 60 is leaf → print [60]
9. Back to 30 → print [30]
10. Both subtrees done → print [10]

Result: 40 → 50 → 20 → 60 → 30 → 10

Python Code:

```
def postorder(node):  
    if node is None: return  
    postorder(node.left)    # left first  
    postorder(node.right)   # right second  
    print(node.data)        # print LAST
```

Property	Value
Time Complexity (All cases)	O(n)
Space Complexity	O(h) — recursion stack
Key Property	Last element printed is always the ROOT
Used for	Deleting trees (safe — children deleted before parent), postfix expression evaluation, dependency resolution

5.4 Level Order Traversal (BFS — Breadth First)

Visit nodes level by level, from top to bottom, left to right within each level. Uses a QUEUE (not recursion) to track which nodes to visit next. Also called BFS (Breadth-First Search) on trees.

Pattern: Visit all nodes at Level 0, then Level 1, then Level 2...

```
Level 0: [10]
Level 1: [20] [30]
Level 2: [40] [50] [60]

Result: 10 → 20 → 30 → 40 → 50 → 60

Uses a QUEUE (not recursion!):
from collections import deque
def level_order(root):
    if not root: return
    queue = deque([root])
    while queue:
        node = queue.popleft()    # dequeue from front
        print(node.data)
        if node.left: queue.append(node.left)    # enqueue left
        if node.right: queue.append(node.right)  # enqueue right

Queue state trace:
Start: [10]
Pop 10: [20, 30]
Pop 20: [30, 40, 50]
Pop 30: [40, 50, 60]
Pop 40: [50, 60]    (40 is leaf, no children added)
Pop 50: [60]
Pop 60: []    done!
```

Property	Value
Time Complexity (All cases)	$O(n)$ — every node visited once
Space Complexity	$O(w)$ — w = maximum width of tree (max nodes at any level)
Space Worst Case	$O(n)$ — complete binary tree, last level can have $n/2$ nodes
Key Property	Finds the SHORTEST PATH from root to any node
Used for	Shortest path problems, level-by-level processing, serialization of trees

5.5 Traversal Comparison Table

Traversal	Order	Data Structure Used	Best Use Case	Output for example tree
Inorder	Left → Root → Right	Recursion / Stack	Sorted output from BST	40 20 50 10 30 60
Preorder	Root → Left → Right	Recursion / Stack	Copy / serialize a tree	10 20 40 50 30 60
Postorder	Left → Right → Root	Recursion / Stack	Delete tree, postfix eval	40 50 20 60 30 10
Level Order	Level by level, L to R	Queue (BFS)	Shortest path, level view	10 20 30 40 50 60

□ 6. Binary Search Tree — Full Implementation

Insert, Search, Delete with all complexities

Complete BST Code (Python)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

# — INSERT —————
# Time: Best O(1), Avg O(log n), Worst O(n) skewed
# Space: O(h) — h = tree height (recursion stack)
    def insert(self, data):
        self.root = self._insert(self.root, data)

    def _insert(self, node, data):
        if node is None:          # base case: empty spot found
            return Node(data)
        if data < node.data:      # smaller → go LEFT
            node.left = self._insert(node.left, data)
        elif data > node.data:    # larger → go RIGHT
            node.right = self._insert(node.right, data)
        return node              # equal = duplicate, ignore

# — SEARCH —————
# Time: Best O(1), Avg O(log n), Worst O(n) skewed
# Space: O(h) — recursion stack
    def search(self, data):
        return self._search(self.root, data)

    def _search(self, node, data):
        if node is None:         # base case: not found
            return False
        if data == node.data:    # found!
            return True
```

```
        elif data < node.data:      # smaller → search LEFT
            return self._search(node.left, data)
        else:                       # larger → search RIGHT
            return self._search(node.right, data)

# — INORDER TRAVERSAL (gives sorted output) —
# Time: O(n), Space: O(h)
def inorder(self):
    result = []
    self._inorder(self.root, result)
    return result

def _inorder(self, node, result):
    if node is None: return
    self._inorder(node.left, result) # left
    result.append(node.data)         # root
    self._inorder(node.right, result) # right

# — FIND MIN / MAX —
# Time: O(h), Space: O(1)
def find_min(self):
    t = self.root
    while t.left: # keep going left
        t = t.left
    return t.data

def find_max(self):
    t = self.root
    while t.right: # keep going right
        t = t.right
    return t.data

# — DELETE —
# Time: Best O(1), Avg O(log n), Worst O(n)
# Space: O(h)
# 3 Cases:
# Case 1: Node is a LEAF → just remove it
# Case 2: Node has 1 CHILD → replace with that child
# Case 3: Node has 2 CHILDREN → replace with inorder successor
#         (smallest value in RIGHT subtree)
def delete(self, data):
    self.root = self._delete(self.root, data)

def _delete(self, node, data):
    if node is None: return None
    if data < node.data:
```

```

        node.left = self.delete(node.left, data)
    elif data > node.data:
        node.right = self._delete(node.right, data)
    else: # found the node to delete
        # Case 1 & 2: 0 or 1 child
        if node.left is None: return node.right
        if node.right is None: return node.left
        # Case 3: 2 children → find inorder successor
        successor = node.right
        while successor.left:
            successor = successor.left
        node.data = successor.data # copy successor value
        node.right = self._delete(node.right, successor.data)
    return node

```

Delete — 3 Cases Visualized

Case 1: Delete LEAF node (no children)

```

Delete 40:  [50]          [50]
            /  \      --> /  \
            [30] [70]    [30] [70]
            /
            [40] <-- just remove it, done!

```

Case 2: Delete node with ONE child

```

Delete 30:  [50]          [50]
            /  \      --> /  \
            [30] [70]    [20] [70]
            /
            [20] <-- replace 30 with its child 20

```

Case 3: Delete node with TWO children

```

Delete 50:  [50]          [60]
            /  \      --> /  \
            [30] [70]    [30] [70]
                    /          /
                    [60]      [65]
                    \
                    [65]

Find inorder successor (smallest in right subtree = 60)
Copy 60 into deleted node's position
Delete 60 from its original position

```

7. Self-Balancing Trees

The solution to skewed trees — always $O(\log n)$

Remember the Skewed Binary Tree problem? If you insert sorted data into a BST, it becomes a linked list and all operations degrade to $O(n)$. Self-balancing trees FIX this by automatically restructuring themselves to maintain $O(\log n)$ height.

7.1 AVL Tree

The first self-balancing BST, invented by Adelson-Velsky and Landis in 1962. Each node tracks a Balance Factor = height(left subtree) - height(right subtree). The rule: Balance Factor must always be -1, 0, or +1. If it goes outside this range, the tree ROTATES to fix itself.

```
Balance Factor = Height(Left) - Height(Right)
Valid values: -1, 0, +1
```

```
Balanced AVL:          Unbalanced (BF = 2 at node 30):
    [30] BF=0           [30] BF=2 ← violation!
    /   \              /
  [20]   [40]         [20]
  BF=0   BF=0         /
                    [10]
```

Fix with RIGHT ROTATION:

```
    [30]                [20]
    /      -->         /   \
  [20]                [10]  [30]
    /
  [10]
```

Now all balance factors are 0 or ±1. Tree is balanced!

Operation	Best	Average	Worst	Space
Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Rotation (rebalance)	$O(1)$	$O(1)$	$O(1)$	$O(1)$

7.2 Red-Black Tree

A self-balancing BST where every node is colored RED or BLACK. 4 rules enforce balance: 1) Root is always black. 2) Red nodes cannot have red children. 3) Every path from root to null leaf has the same number of black nodes. 4) Null leaves are considered black.

Red-Black Tree Example:

```

      [10 BLACK]
     /      \
  [5 RED]   [20 RED]
  /  \     /  \
[3 BLK][7 BLK][15 BLK][25 BLK]

```

Rules verified:

- ✓ Root (10) is BLACK
- ✓ Red nodes (5, 20) have only BLACK children
- ✓ Every path from root to null has 2 black nodes

Red-Black vs AVL:

AVL: Stricter balance → faster LOOKUPS

Red-Black: Looser balance → faster INSERTIONS/DELETIONS

That is why Linux kernel uses Red-Black for frequent inserts!

Operation	Best	Average	Worst	Space
Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
Insert	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
Delete	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$

□ Who Uses AVL and Red-Black Trees?

Company / System	Which Tree	How
Linux Kernel	Red-Black Tree	Process scheduling, virtual memory areas, timer management
Java	Red-Black Tree	TreeMap, TreeSet — the sorted collections in Java SDK
C++ STL	Red-Black Tree	std::map and std::set — every C++ program uses these
PostgreSQL	B-Tree (AVL variant)	Database index structure for fast query execution
Nginx	Red-Black Tree	Timer management for handling request timeouts
MongoDB	B-Tree	WiredTiger storage engine for document indexing

8. Heap — The Priority Master

Complete BT that always keeps the min or max at top

A Heap is a Complete Binary Tree with one extra rule: the parent is always greater than (Max-Heap) or less than (Min-Heap) its children. The ROOT always holds the MAXIMUM (max-heap) or MINIMUM (min-heap) value — accessible in $O(1)$.

MAX-HEAP (parent $\geq$ children): MIN-HEAP (parent $\leq$ children):



Max-Heap rule: $90 > 70$, $90 > 80$, $70 > 50$, $70 > 60$ etc. ✓

Min-Heap rule: $5 < 10$, $5 < 8$, $10 < 20$, $10 < 15$ etc. ✓

KEY: Stored as an ARRAY (array representation of complete BT!)

Max-Heap array: [90, 70, 80, 50, 60, 40, 75]

Index: 0 1 2 3 4 5 6

Parent of index $i = (i-1)//2$

Children of i : left= $2i+1$, right= $2i+2$

Operation	Best	Average	Worst	Space	Note
Get Max/Min (peek)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Root is always max or min!
Insert (heappush)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Add at end, bubble up
Delete Max/Min (heappop)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	Remove root, replace with last, sift down
Build heap from array	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Better than inserting one by one!
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	In-place sorting using heap

Python Heap (Min-Heap by default):

```
import heapq
```

```
heap = []
```

```
heapq.heappush(heap, 30)    # insert 30
heapq.heappush(heap, 10)    # insert 10
heapq.heappush(heap, 20)    # insert 20
print(heap[0])              # 10 — minimum always at index 0! O(1)
print(heapq.heappop(heap))  # 10 — remove min O(log n)
print(heapq.heappop(heap))  # 20 — next min
```

For Max-Heap: negate the values

```
heapq.heappush(heap, -30)   # store as -30
heapq.heappush(heap, -10)
print(-heap[0])             # 30 — max value
```

□ Who Uses Heaps?

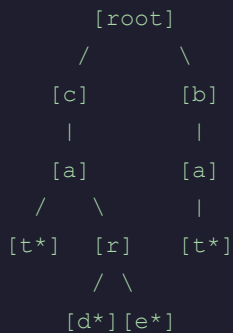
Company / System	Use Case
Google Maps	Dijkstra's shortest path — min-heap picks the closest unvisited node
Amazon AWS	Task scheduler — min-heap picks the highest priority (lowest timestamp) task
Operating Systems	CPU process scheduler — priority queue implemented as a heap
Python	heapq module — used in every Python application needing a priority queue
Java	PriorityQueue class — Android apps, Spring Boot servers use this extensively
Netflix	Video recommendation queue — content sorted by relevance score using a heap

9. Trie — The Autocomplete Tree

Specialized tree for text search and autocomplete

A Trie (pronounced 'try') is a tree where each node represents a CHARACTER. Words are stored letter by letter from root to leaf. Every path from root to a marked node spells out a complete word. This makes prefix-based search lightning fast.

Words stored: 'cat', 'car', 'card', 'care', 'bat'



* = end of a word

'cat' = root → c → a → t*

'car' = root → c → a → r

'card' = root → c → a → r → d*

'care' = root → c → a → r → e*

'bat' = root → b → a → t*

Search 'car': $O(\text{length of word}) = O(3)$ — NOT $O(n)$ total words!

Prefix search 'ca': Return all words starting with 'ca' instantly

Operation	Best	Average	Worst	Space	Note
Insert word	$O(L)$	$O(L)$	$O(L)$	$O(L)$	L = length of word. Constant per character
Search word	$O(1)$	$O(L)$	$O(L)$	$O(1)$	Best = word not in trie (short miss)
Prefix search	$O(P)$	$O(P)$	$O(P)$	$O(1)$	P = length of prefix. Extremely fast!
Delete word	$O(L)$	$O(L)$	$O(L)$	$O(1)$	Mark end node,

Operation	Best	Average	Worst	Space	Note
					optionally clean up
Space for n words	-	-	-	$O(N*L)$	N=words, L=avg length. Can be large

□ Who Uses Tries?

Company	Use Case
Google	Search autocomplete — as you type, trie finds all matching prefixes in milliseconds
Amazon	Product search autocomplete — 'lap' instantly suggests 'laptop', 'lapdesk', etc.
Spotify	Song and artist search — prefix matching for instant search results
Microsoft	Word's autocorrect and autocomplete — dictionary stored as a trie
Phone Contacts	Contact search by name — every smartphone uses a trie for contact lookup
DNA Sequencing	Bioinformatics — DNA sequences (ACGT) stored in tries for fast pattern matching

⚡ 10. Master Complexity Reference

Every tree operation at a glance

Tree Type	Search Best	Search Avg	Search Worst	Insert Avg	Delete Avg	Space
Binary Tree	O(1)	O(n)	O(n)	O(n)	O(n)	O(n)
BST (balanced)	O(1)	O(log n)	O(n)*	O(log n)	O(log n)	O(n)
AVL Tree	O(1)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Red-Black Tree	O(1)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Heap (get min/max)	O(1)	O(1)	O(1)	O(log n)	O(log n)	O(n)
Trie	O(L)	O(L)	O(L)	O(L)	O(L)	O(N*L)
Complete BT	O(1)	O(n)	O(n)	O(log n)	O(log n)	O(n)

* BST worst case is O(n) when tree is skewed (sorted input). Use AVL or Red-Black to prevent this.

Traversal	Time	Space (balanced)	Space (skewed)	Key Use
Inorder	O(n)	O(log n)	O(n)	Sorted output from BST
Preorder	O(n)	O(log n)	O(n)	Copy / serialize tree
Postorder	O(n)	O(log n)	O(n)	Delete tree, postfix eval
Level Order (BFS)	O(n)	O(w) — max width	O(1)	Shortest path, level view

11. Big Tech — Who Uses What Tree Where

Real production use cases across the industry

Company	Tree Type	Where Exactly
Google	Trie	Search autocomplete — prefix matching for billions of queries/day
Google	B-Tree	Bigtable and Spanner database indexes
Google Maps	Min-Heap + BST	Dijkstra's algorithm for routing — heap picks next closest node
Meta (Facebook)	Generic Tree	React's Virtual DOM — component hierarchy
Meta	BST	Social graph traversal for friend suggestions and mutual connections
Amazon	Min-Heap	AWS SQS priority queue — task scheduling by timestamp
Amazon	B-Tree	DynamoDB — partition key range queries use tree indexes
Microsoft	Red-Black Tree	Windows NTFS file system — directory management
Microsoft	BST (AVL)	.NET's SortedDictionary — sorted key-value lookup
Apple	B-Tree	HFS+ and APFS file systems — file metadata index
Apple	Trie	Siri — prefix search for contacts, songs, apps
Linux Kernel	Red-Black Tree	CFS process scheduler, virtual memory area management, timers
MongoDB	B-Tree	WiredTiger storage engine — all collection indexes
PostgreSQL	B-Tree	Default index type — powers almost every SQL query with WHERE
Redis	Skip List (tree-like)	Sorted Sets — leaderboards, time-series, range queries
Nginx	Red-Black Tree	Timer management — request timeout tracking

Company	Tree Type	Where Exactly
Chrome Browser	Generic Tree	DOM tree — every webpage's HTML structure
Spotify	Trie	Search autocomplete for songs, artists, playlists
Netflix	Heap	Content recommendation ranking — priority queue of suggested content
Uber	Min-Heap + BFS	Pathfinding and driver matching — graph + heap for shortest paths

12. Interview Cheat Sheet

The patterns that appear in 90% of tree interview questions

Most Common Interview Patterns

Pattern 1: Tree Traversal Questions

Q: Print all nodes in sorted order → Inorder traversal of BST

Q: Clone/copy a tree → Preorder traversal

Q: Delete all nodes → Postorder traversal

Q: Find the level of a node → Level order (BFS)

Pattern 2: BST Validation

Q: Is this a valid BST?

Strategy: Pass min and max bounds down recursively.

Every node must satisfy: $\text{min} < \text{node.value} < \text{max}$

Time: $O(n)$, Space: $O(h)$

Pattern 3: Height / Depth Problems

Q: Find height of a tree

Q: Is the tree balanced?

Strategy: Use postorder — compute heights bottom up.

$\text{height}(\text{node}) = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$

Time: $O(n)$, Space: $O(h)$

Pattern 4: Lowest Common Ancestor (LCA)

Q: Find the lowest common ancestor of nodes A and B

Strategy: Recurse. If current node == A or B, return it.

If both left and right return non-null, current node is the LCA.

Time: $O(n)$, Space: $O(h)$

Pattern 5: Path Sum Problems

Q: Does any root-to-leaf path sum equal target?

Strategy: Recursively subtract node value from target.

Base case: at leaf, check if remaining == 0.

Time: $O(n)$, Space: $O(h)$

Key Things to Remember in Interviews

- Always ask: is the tree a BST? If yes, you can use binary search — $O(\log n)$ instead of $O(n)$.
- Always ask: is the tree balanced? Unbalanced = skewed = $O(n)$ worst case for BST operations.
- DFS (Inorder/Pre/Post) uses recursion or a stack. BFS (Level Order) uses a queue.
- Space complexity of recursion = $O(\text{height})$. Balanced tree: $O(\log n)$. Skewed: $O(n)$.
- Inorder traversal of a BST = sorted array. This fact solves many interview problems.
- When in doubt: try recursion. Trees and recursion are made for each other.
- For kth smallest/largest in BST: inorder traversal and count. Time $O(n)$, Space $O(h)$.
- For any 'max/min path' problem: use postorder and pass values upward.

□ 13. Quick Summary

Everything in one place

Tree Type	Children	Rule	Best For
Binary Tree	0, 1, or 2	No extra rule	General hierarchy, expression trees
BST	0, 1, or 2	Left < Parent < Right	Sorted data, fast search
Strict BT	0 or 2 only	Never exactly 1 child	Huffman coding, expression trees
Complete BT	0, 1, or 2	Left-to-right fill, level-by-level	Heaps, priority queues
Skewed BT	0 or 1 only	All nodes in one direction	Avoid — worst case performance
Degenerate BT	0 or 1 only	Zigzag direction	Avoid — O(n) everything
Extended BT	0 or 2 only	Add dummies to make strict	Huffman analysis, compiler theory
Generic Tree	Any number	No child count limit	File systems, DOM, org charts
AVL Tree	0, 1, or 2	Balance factor in {-1,0,1}	Fast lookups, read-heavy workloads
Red-Black Tree	0, 1, or 2	Color rules ensure log n height	Fast inserts/deletes (Java, Linux)
Heap	0, 1, or 2	Parent >= children (max) or <=	Priority queues, Dijkstra
Trie	0 to 26 (alphabet)	Each edge = one character	Autocomplete, spell check, search

Master Trees. Master Interviews. □

Trees are not just a data structure — they are the architecture of the modern world.

From Google Search to Linux kernel, from your file system to every database query — trees are everywhere.